
Build Solution

Given a use case, identify the factors needed to build resilience in an integration solution for system updates.

Develop your career with Salesforce Training and Certification Preparation

Table of Contents



Business Scenario

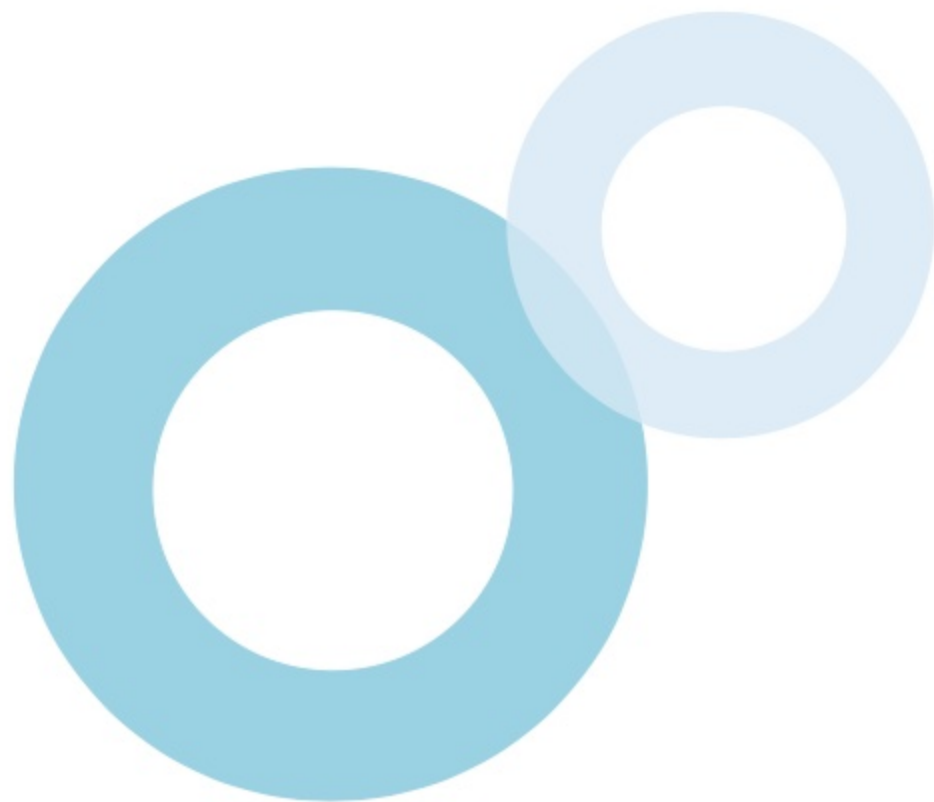


Requirements & Solutions



After studying this topic, you should be able to:

-  Identify the key factors that should be considered for building resilience in different types of integration solutions.
-  Given various business requirements, describe how to build resilient integration solutions or make existing integration solutions resilient.



Introduction

This topic describes the key factors that should be identified to build **resilient integration solutions**. Different factors need to be considered for different integration patterns. For instance, for building resilience in Remote Process Invocation - Request & Reply solutions, **recovery, timeliness, and state management** should be considered. Changes should not be committed until a successful response has been received from the remote system. **Retries** and **configurable timeout** should be utilized for Apex callouts if necessary.

Ongoing state tracking using keys can be considered for various integration solutions. For example, when using **batch data synchronization**, Salesforce can store the **unique ID** returned by the remote system, and prior to performing any operations in the future, the ID can be checked to avoid record duplication.

Reliable messaging is also an important consideration for ensuring resilience in certain types of integration solutions. For example, when using REST API or SOAP API, the remote system should implement a **reliable messaging system** for timeout and error handling scenarios.



Resilient Integration Solutions

Request & Reply Solutions

For Remote Process Invocation - Request & Reply solutions, factors related to recovery, timeliness, and state management need to be considered for building resilience.



Fire & Forget Solutions

For Remote Process Invocation - Fire & Forget solutions, factors related to recovery, timeliness, state management, and reliable messaging need to be considered for building resilience.



Batch Data Synchronization

For Batch Data Synchronization solutions, factors related to recovery, timeliness, state management, and middleware capabilities need to be considered for building resilience.



Remote Call-In Solutions

For Remote Call-In solutions, factors related to recovery, timeliness, state management, and reliable messaging need to be considered for building resilience.



Remote Process Invocation - Request & Reply

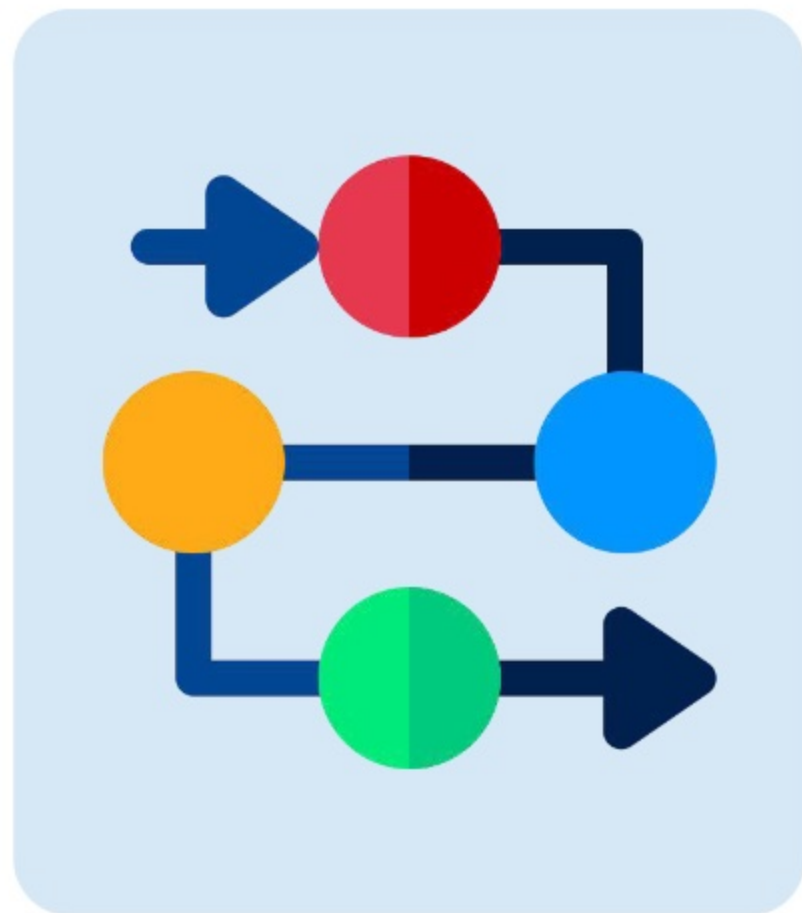
For **Remote Process Invocation - Request & Reply** solutions, factors related to **recovery**, **timeliness**, and **state management** need to be considered for building resilience.

❖ RECOVERY

Changes should **not be committed** to Salesforce until a successful response is received. The operation should be **retried** if necessary.

❖ TIMELINESS

Apex timeouts have a configurable timeout of up to **120 seconds**. If the request is invoked from the user interface, the process must not keep the user waiting. **Apex synchronous transaction governor limits** must be considered. The risk of instantiating **more than 10 transactions** that run for more than **5 seconds** each should be mitigated.

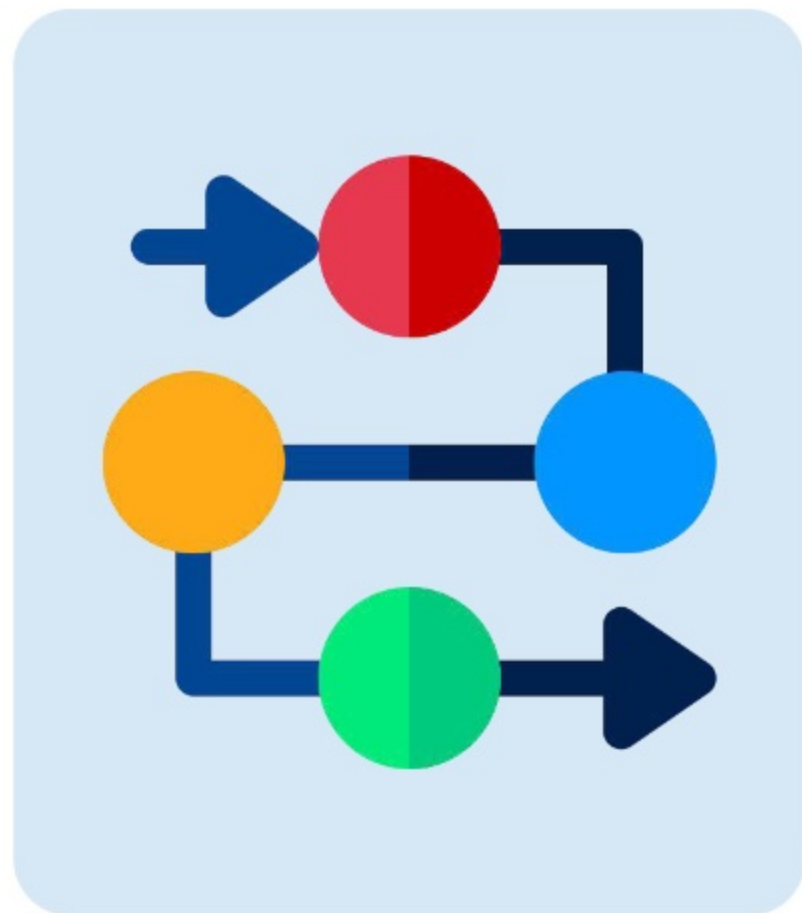


Remote Process Invocation - Request & Reply

For **Remote Process Invocation - Request & Reply** solutions, factors related to **recovery**, **state management**, and **timeliness** need to be considered for building resilience.

❖ STATE MANAGEMENT

Keys can be used for **ongoing state tracking**. If **Salesforce** is the master system, the **remote system** can store either the **Salesforce record Id** or **some other unique surrogate key**. If the **remote system** is the master, Salesforce can store the **unique key** returned from the remote system.



Remote Process Invocation - Fire & Forget

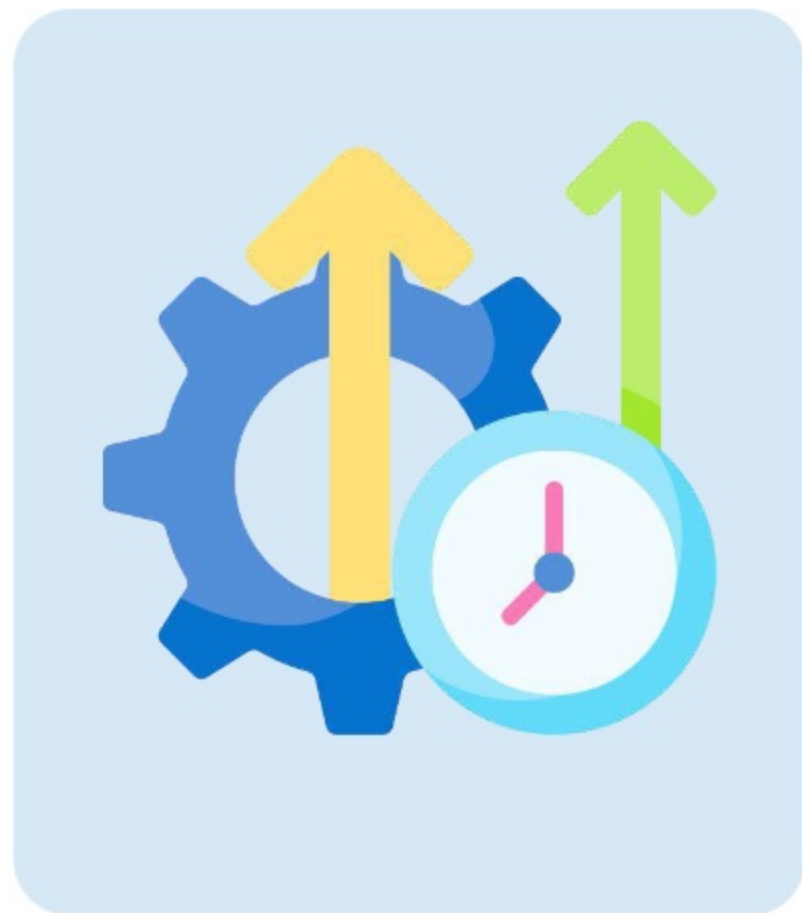
For **Remote Process Invocation - Fire & Forget** solutions, factors related to **recovery**, **timeliness**, **state management**, and **reliable messaging** need to be considered for building resilience.

❖ RECOVERY

For **Apex callouts**, a **custom retry mechanism** must be created if required. For **outbound messaging**, Salesforce **automatically retries** if no positive acknowledgement is received within the timeout period, up to **24 hours**. For **platform events**, the **remote system must initiate retries**.

❖ TIMELINESS

When using **Outbound Messaging**, **positive acknowledgement** must occur within **24 hours**; otherwise, the message expires. When using platform events, Salesforce sends the events to the event bus and **doesn't wait for acknowledgement**. **High-volume event messages** are stored for **72 hours**. The subscriber can request to replay the event using the **replay ID**. A **CometD client** can subscribe to a channel to retrieve past events.

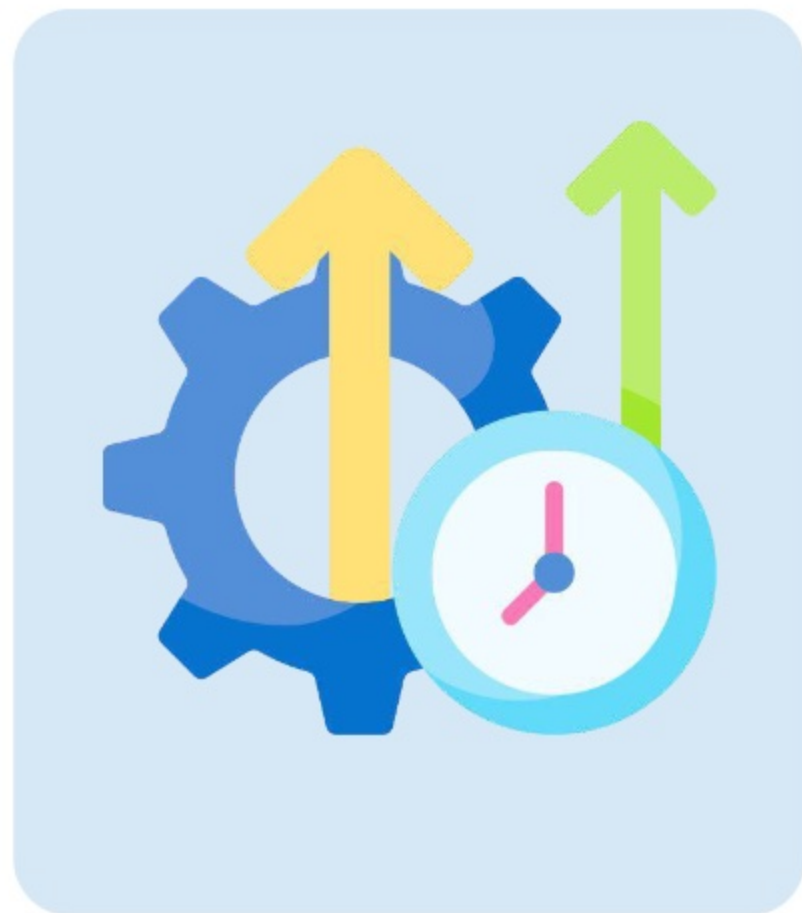


Remote Process Invocation - Fire & Forget

For **Remote Process Invocation - Fire & Forget** solutions, factors related to **recovery**, **idempotency**, and **state management** need to be considered for building resilience.

❖ STATE MANAGEMENT

Keys can be used for **ongoing state tracking**. If **Salesforce** is the master system, the **remote system** can store either the **Salesforce record Id** or **some other unique surrogate key**. If the **remote system** is the master, Salesforce can provide a unique ID in the call to the remote process, and then the remote system can perform a callback to Salesforce to update the record with a unique identifier using the **unique ID**.

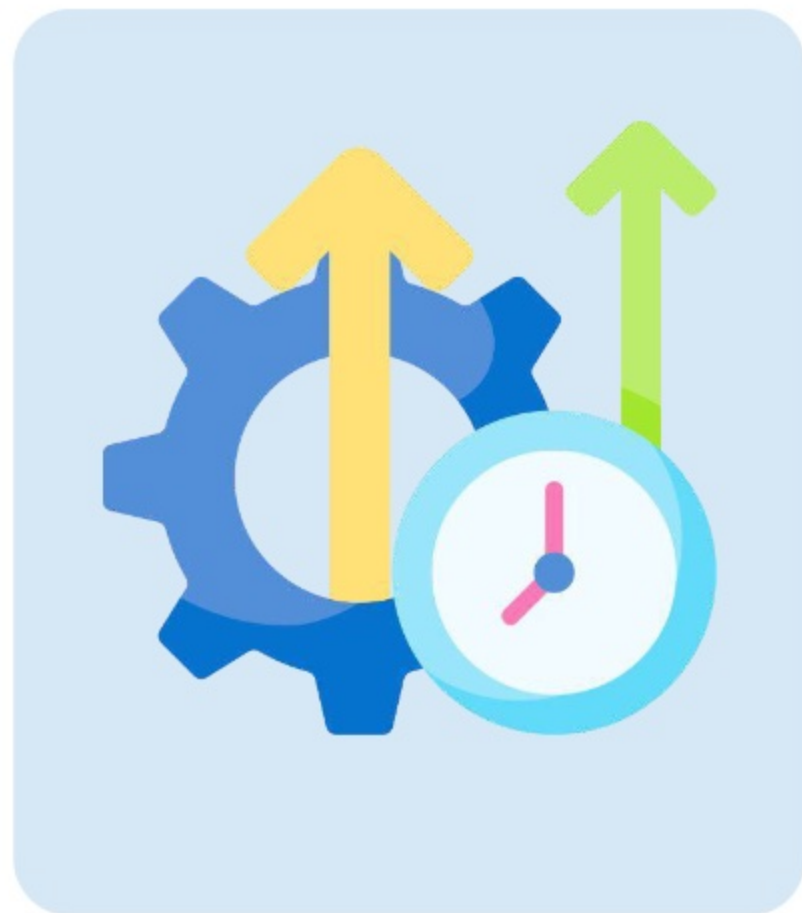


Remote Process Invocation - Fire & Forget

For **Remote Process Invocation - Fire & Forget** solutions, factors related to **recovery**, **idempotency**, and **state management** need to be considered for building resilience.

❖ RELIABLE MESSAGING

For **Apex callouts**, processing of a **positive acknowledgement from the remote system** and **custom retry logic** can be implemented in Salesforce for reliable messaging. The remote system can also implement a reliable messaging system, such as **JMS or MQ**. **Outbound messaging** provides reliable messaging through **automatic retries for up to 24 hours**, which can be extended to **seven days** by a request to Salesforce support. **Platform Events** provides reliable messaging by **temporarily persisting event messages** in the event bus. The subscriber can **replay missed messages using the replay ID**.

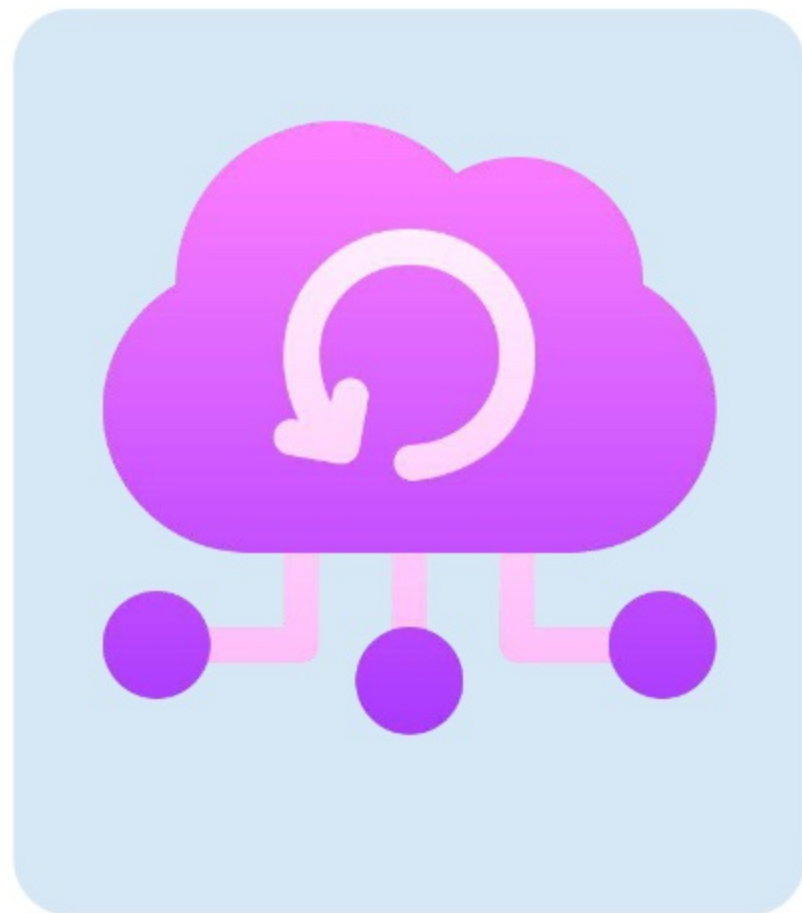


Batch Data Synchronization

For **Batch Data Synchronization** solutions, factors related to **recovery**, **timeliness**, **state management**, and **middleware capabilities** need to be considered for building resilience.

❖ RECOVERY

When reading data from Salesforce using **Change Data Capture**, the remote system must initiate retries using the replay ID. When **reading data from Salesforce** or **writing to Salesforce** using a **third-party ETL tool**, the ETL process should be restarted to recover from a failed read operation. If the operation succeeds but there are failed records, an **immediate restart** or **subsequent execution** of the job should be considered. A **delayed restart** could be used to **allow time to triage** and **correct data** that might be causing errors.



Batch Data Synchronization

For **Batch Data Synchronization** solutions, factors related to **recovery**, **timeliness**, **state management**, and **middleware capabilities** need to be considered for building resilience.

❖ TIMELINESS

The **source and target systems** should be insulated during batch processing windows. Loading batches during business hours can cause **data contention**, which can cause a **user's update failing** or a **batch load (or partial batch load) failing**. Data contention can be avoided through **data segmentation** using **record types** and other **filtering criteria**.

❖ STATE MANAGEMENT

Surrogate keys can be used between the two systems to implement state management. The **Remote Call-In** pattern can be used for any type of **transaction management** across Salesforce entities.

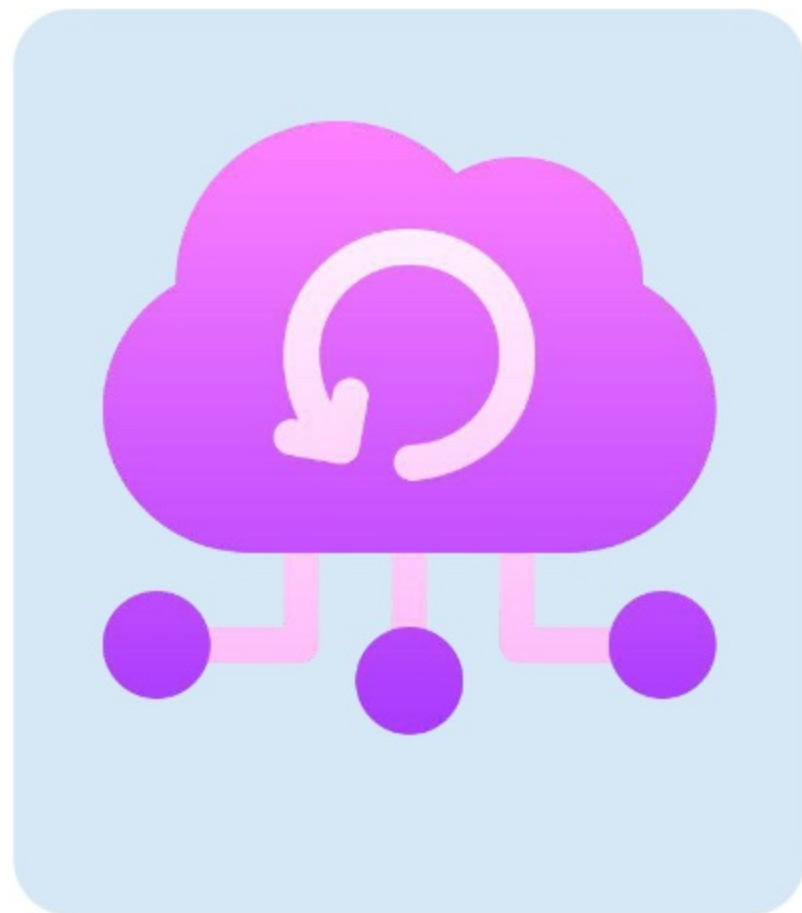


Batch Data Synchronization

For **Batch Data Synchronization** solutions, factors related to **recovery**, **timeliness**, **state management**, and **middleware capabilities** need to be considered for building resilience.

❖ MIDDLEWARE

The middleware tool should be able to **extract, transform, and load** data. It should support the **Salesforce Bulk API**, and ideally the **getUpdated()** function, which is the closest implementation of the standard change data capture capability. Desirable features include **encryption, signing, reliable delivery, transaction management, routing**, etc.



Remote Call-In

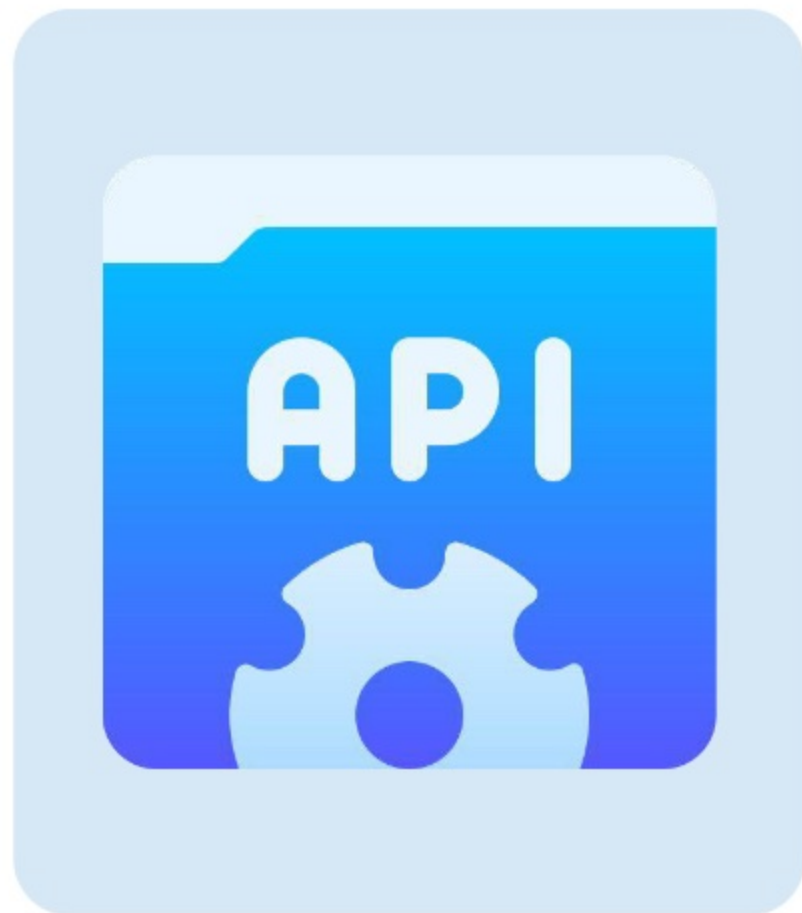
For **Remote Call-In** solutions, factors related to **recovery**, **timeliness**, **state management**, and **reliable messaging** need to be considered for building resilience.

❖ RECOVERY

A **custom retry mechanism** that ensures **idempotent design characteristics** can be built. When using **platform events**, subscribers can use the **replay ID** to fetch messages within a certain time period.

❖ TIMELINESS

Session timeout and **query timeout** apply to **SOAP API** and **Apex web service API**. The session times out when there's no activity. Each SOQL query has a timeout limit of **120 seconds**.



Remote Call-In

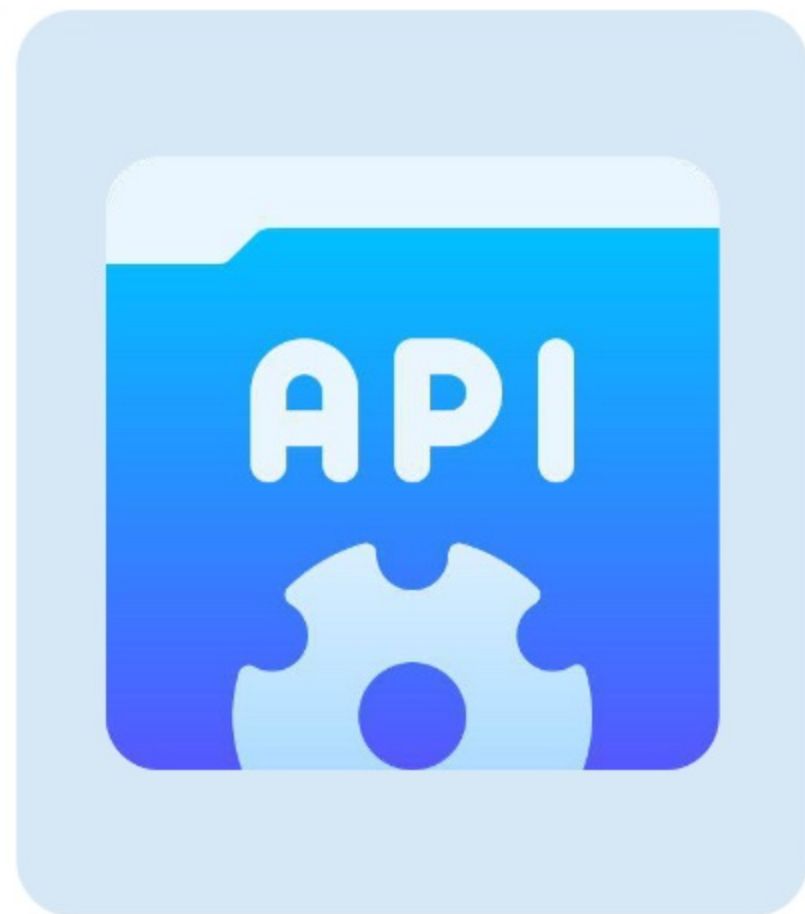
For **Remote Call-In** solutions, factors related to **recovery**, **timeliness**, **state management**, and **reliable messaging** need to be considered for building resilience.

❖ STATE MANAGEMENT

Keys can be used for **ongoing state tracking**. If **Salesforce** is the master system, the **remote system** can store either the **Salesforce record Id** or **some other unique surrogate key**. If the **remote system** is the master, Salesforce can store the **unique key** returned from the remote system.

❖ RELIABLE MESSAGING

The remote system should implement a reliable messaging system to **manage error and timeout scenarios** successfully. **REST API and SOAP API** do not provide explicit support for any reliable messaging protocols.

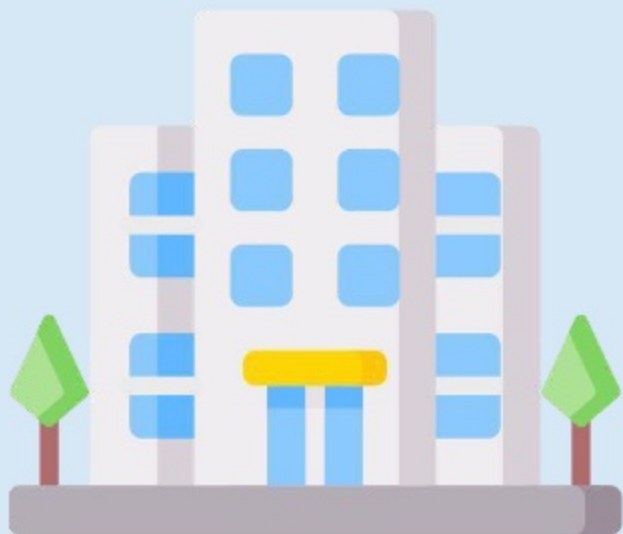


Business Scenario

Table of Contents 

Business Scenario

Read the business scenario below and determine the most appropriate solutions for the company's requirements.



Cosmic Belle is an Australian B2B supplier of beauty products. It sells its products to pharmacies throughout the country. There are more than 10,000 pharmacy stores that regularly place orders on the company's B2B Commerce Portal. On average, hundreds of orders are placed every day.

Salesforce provides customer portal and engagement functionality and is used for managing accounts, contacts, leads, and opportunities. The system landscape also comprises an ERP system, which is the source of truth for employee data, products, and pricebooks. It is mainly used for the processing and fulfillment of orders.

Requirements & Solutions

[Table of Contents](#)



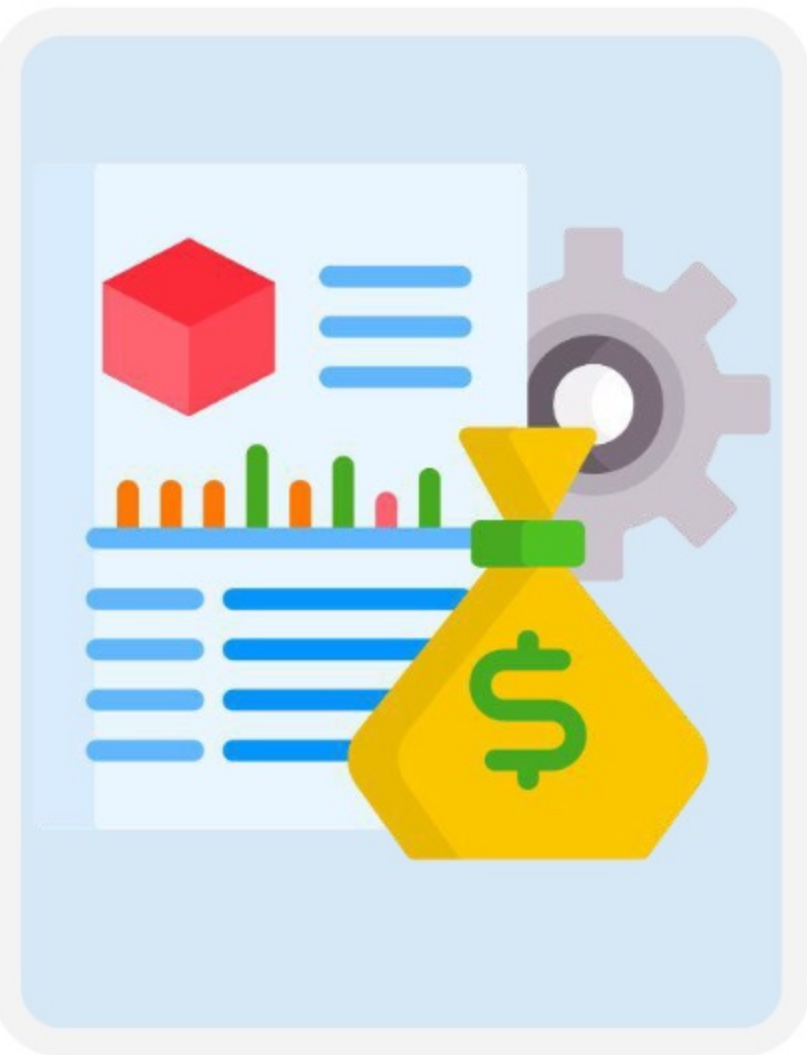
Requirement 1



SCENARIO

Orders placed by customers on the Salesforce B2B Commerce Portal are added to the ERP system for fulfillment and processing. Order records are currently extracted from Salesforce and inserted into the ERP system every night through an existing batch data synchronization job. The integration uses an INSERT statement in SQL to load the orders into the ERP system. The company already has an ETL Tool that is used for the current integration and would prefer using the same tool moving forward.

However, the ERP system often goes offline for system updates, which causes the integration to fail. The IT team manually inserts the unsuccessful orders in the morning instead of using a full synchronization load which would create duplicate orders. The solution architect of the company needs to recommend an approach that can be used to self-correct the integration during the next data load.



Solution 1

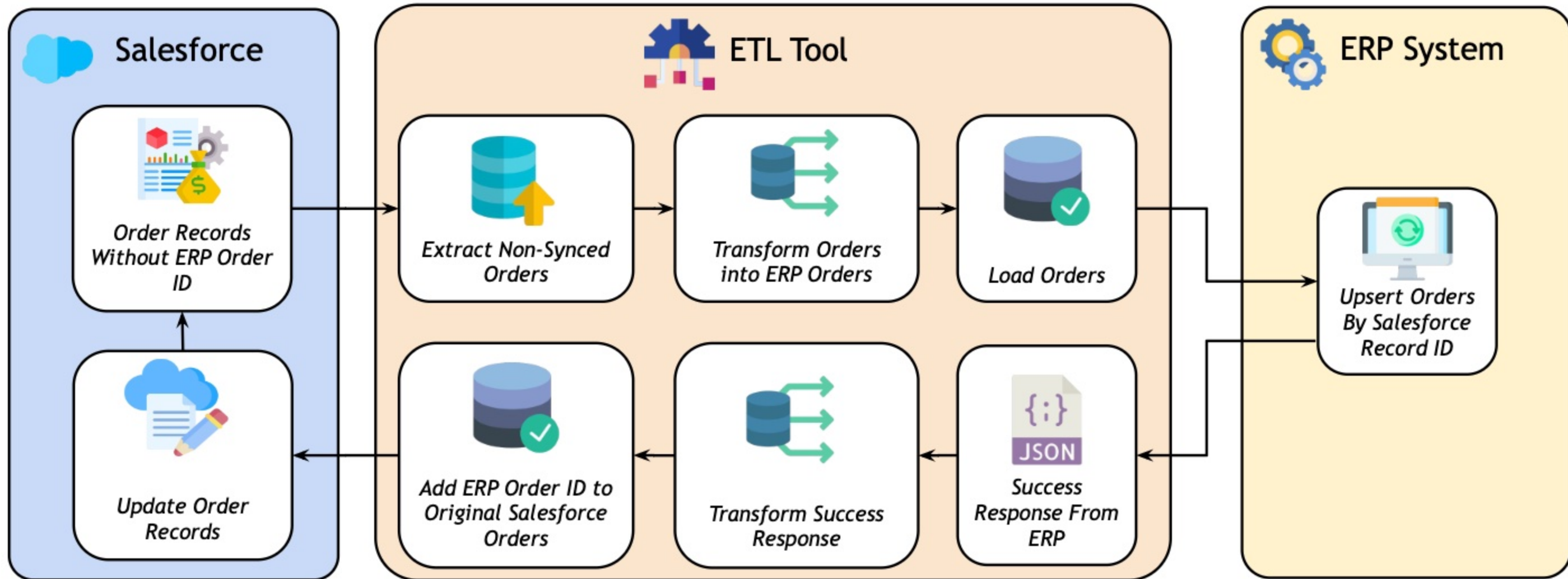
SOLUTION



The '**Batch Data Synchronization**' pattern can continue to be utilized using the ETL tool to load orders into the ERP system every night. A custom field called '**ERP Order ID**' can be created on the Order object in Salesforce to store the order ID generated by the ERP system. To ensure that only non-synced orders are extracted, the integration can be updated to extract only the orders without a populated '**ERP Order ID**'. To avoid duplicate orders in the ERP system, the **UPSERT** statement can be used in place of the INSERT statement to update existing orders that were inserted successfully and insert new orders in a single call. Once the UPSERT operation is complete, the ETL can process the response obtained from the ERP system and update the '**ERP Order ID**' field on the corresponding order records in Salesforce. These changes would ensure that, when a nightly batch data synchronization job fails due to offline status, the next synchronization job automatically syncs the orders that were not loaded previously.



Solution Diagram



Requirement 2



SCENARIO

A Lightning web component is used by sales users to create or update account information in Salesforce. There are two separate actions in the Lightning web component. Once an account is created or updated by a user, a separate button is used to perform a synchronous REST callout to the ERP system to perform the same operation based on the entered information. A unique ID returned by the ERP system is stored on the account record in Salesforce.

However, the ERP system often returns an error due to request timeout or server unavailability. The user has to retry the callout several times by clicking the button repeatedly. Sometimes a user skips the callout due to the error, which results in data inconsistency. The solution architect needs to recommend a suitable approach that ensures the resilience of the integration and proper account data synchronization between the two systems.



Solution 2



SOLUTION

It is a best practice to ensure that changes are not committed to Salesforce unless a successful response is received from the external system. The process used in the integration solution can be improved by ensuring that the **REST callout** to the ERP system is performed **before the DML operation (account insertion or update)** in Salesforce. The two existing buttons can be replaced with a **single button in the Lightning web component** for the action performed by the current user. This button can be clicked by the user to perform the callout to the ERP system, and once a successful response is received from the ERP system, the **DML operation** can be performed in Salesforce automatically using a **separate Apex transaction**. A **custom retry mechanism** can be implemented in Apex for the REST callout to the ERP system. If an error occurs, the callout can be **retried** up to a **certain number of times**. Also, a **custom timeout** can be used for the callout to ensure that the request does not time out. Salesforce allows using a timeout value of up to **120 seconds**.



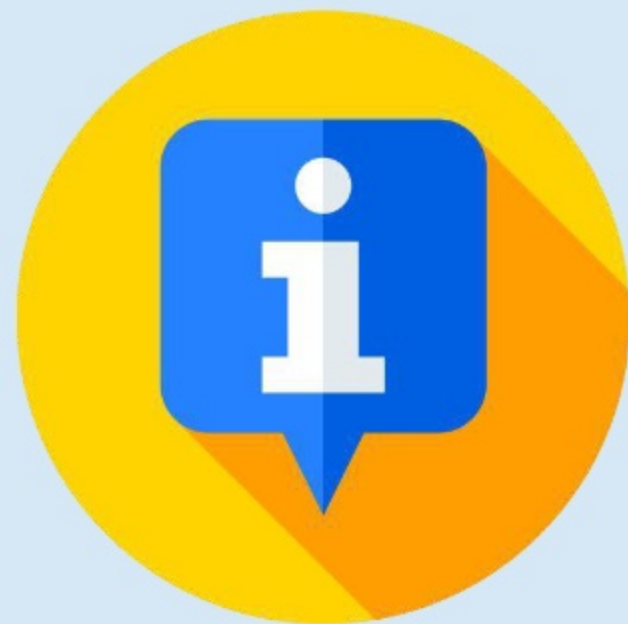
Requirement 3



SCENARIO

When a new product is created in Salesforce, a record-triggered flow publishes a platform event message containing information about the product. The ERP system has a CometD client that has subscribed to receive platform event messages. When a new event message is received, a product record is automatically created in the ERP system.

However, the ERP system often misses an event message due to connection failure, which results in a missing product record in the system. There is currently no mechanism to retrieve missed events. The solution architect needs to recommend a suitable solution for this use case.



Solution 3



SOLUTION

To ensure that missed platform event messages do not result in missing product records, the **ReplayId** field value can be used by the CometD client to replay any events **within the retention window** or after a certain event message, for example, **after a connection failure**. The CometD client can store a **replay ID** and use it on **resubscription** to retrieve missed events that are within the retention window. New product records can be created after retrieving the missed events. **High-volume event messages** are stored for **72 hours** in the event bus.



Requirement 4



SCENARIO

When a performance review record is created for a particular employee in the ERP system, a corresponding review record should be created for the employee in Salesforce automatically. An Apex web service is being developed in Salesforce for this use case. A custom process in the ERP system can call the web service automatically when a new performance review record is created. However, the developer is concerned about the possibility of errors and record duplication when the web service is called by the custom process. The solution architect needs to recommend a suitable approach for building a resilient solution for this requirement.



Solution 4



SOLUTION

To build a resilient solution, a **retry mechanism** should be added to the custom process in the ERP system that calls the **Apex web service**. The web service should be updated to ensure that it can handle retries by the custom process. Furthermore, the custom process should be built with **idempotent design characteristics** to ensure that duplicate performance review records are not created in Salesforce. The record in Salesforce can store a **unique ID** from the ERP system. The custom process can first check for the existence of a record using a **newly generated unique ID** by invoking a method in the Apex web service. If a record with the unique ID is **not found**, a **new record** can be created.



Learn More



[Integration Patterns and Practices](#)



[Application Integration Patterns for Salesforce Lightning Platform](#)